

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

***CO1:** Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 50%

Dr. G. A. Senthil

Code of Conduct:

I __T Bhanu Teja -192472259__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

T Bhanu Teja

QUESTIONS: 5

Total Marks: 50

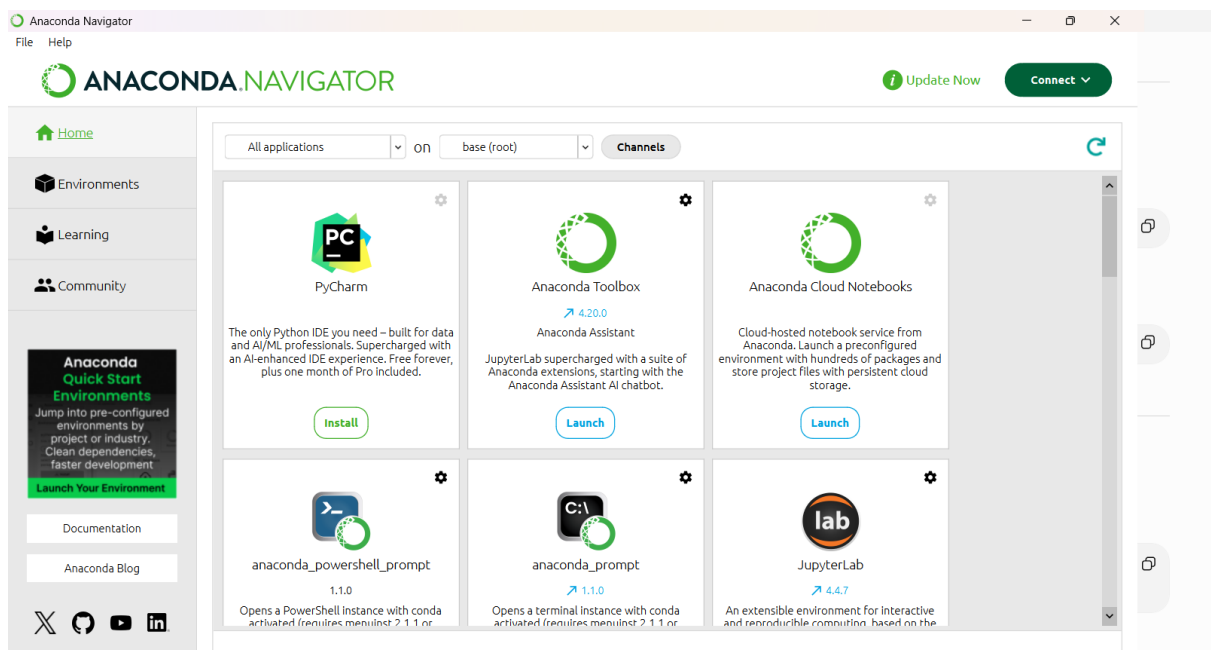
Annexure A

RL Basic Experiments demo with Keras and TensorFlow using Anaconda navigator

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Installation of Anaconda:



Installation of Py,tensorflow,Keras,Gymnasium,Numpy,Matplot lib:

```
C:\Users\bhanu>py --version
Python 3.13.14

C:\Users\bhanu>py -3.13 -m pip install nltk
Requirement already satisfied: nltk in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (3.10.0)
Requirement already satisfied: defusedxml in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from nltk) (0.7.1)
Requirement already satisfied: click in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from nltk) (8.4.2)
Requirement already satisfied: joblib in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from nltk) (2026.6.28)
Requirement already satisfied: tqdm in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from nltk) (4.68.4)
Requirement already satisfied: colorama in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from click->nltk) (0.4.6)
```

```
C:\Users\bhanu>py -3.13 -m pip install numpy
Requirement already satisfied: numpy in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (2.5.1)

C:\Users\bhanu>py -3.13 -m pip install matplotlib
Requirement already satisfied: matplotlib in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (3.11.0)
Requirement already satisfied: contourpy>=1.0.1 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.25 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib) (2.5.1)
Requirement already satisfied: packaging>=20.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=9 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

```
C:\Users\bhanu>py -3.13 -m pip install tensorflow
Collecting tensorflow
  Downloading tensorflow-2.21.0-cp313-cp313-win_amd64.whl.metadata (4.5 kB)
Collecting absl-py>=1.0.0 (from tensorflow)
  Downloading absl_py-2.5.0-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Downloading astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=25.9.23 (from tensorflow)
  Downloading flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 (from tensorflow)
  Downloading gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta>=0.1.1 (from tensorflow)
  Downloading google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow)
  Downloading libclang-18.1.1-py2.py3-none-win_amd64.whl.metadata (5.3 kB)
Collecting opt_einsum>=2.3.2 (from tensorflow)
  Downloading opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: packaging in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from tensorflow) (26.2)
Collecting protobuf<8.0.0,>=6.21.1 (from tensorflow)
```

Verify the Installation

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

```
print("All Libraries Installed Successfully!")
```

Execute a Simple Reinforcement Learning Program

```
import gymnasium as gym

env = gym.make("CartPole-v1")

state, info = env.reset()

print("Initial State:")
print(state)

env.close()
```

Expected Output

```
Initial State:
[ 0.02 -0.01  0.03  0.04
```

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Procedure / Algorithm

1. Import the Gymnasium library.
2. Create a Gymnasium environment (CartPole/FrozenLake/MountainCar).
3. Reset the environment to get the initial state.
4. Display the current state.
5. Select an action (random action).
6. Execute the action and observe the next state and reward.
7. Repeat until the episode ends.
8. Close the environment.

Python Program

```
import gymnasium as gym

# Create CartPole environment
env = gym.make("CartPole-v1")

# Reset environment
state, info = env.reset()

print("Initial State:", state)

# Run for 5 steps
for i in range(5):
    action = env.action_space.sample() # Random action
    state, reward, terminated, truncated, info = env.step(action)

    print(f"\nStep {i+1}")
    print("Action:", action)
    print("State:", state)
```

```
print("Reward:", reward)
```

```
if terminated or truncated:
```

```
    break
```

```
env.close()
```

Sample Output

Initial State:

[0.03 -0.01 0.02 0.04]

Step 1

Action: 1

State: [0.04 0.18 0.03 -0.25]

Reward: 1.0

Step 2

Action: 0

State: [0.03 0.00 0.02 0.05]

Reward: 1.0

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Algorithm

1. Import the required libraries.
2. Create the RL environment.
3. Reset the environment to obtain the initial state.
4. Select an action.
5. Execute the action in the environment.
6. Receive the next state and reward.
7. Repeat until the episode ends.
8. Close the environment.

Python Program

```
import gymnasium as gym
# Create the CartPole environment
env = gym.make("CartPole-v1")
state, info = env.reset()
done = False
total_reward = 0
while not done:
    action = env.action_space.sample() # Random action
    state, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    done = terminated or truncated
print("Total Reward:", total_reward)
env.close()
```

Sample Output

Total Reward: 18.0

Result

The Reinforcement Learning framework was successfully implemented. The agent interacted with the environment by performing actions and receiving rewards until the episode terminated.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Introduction

A **Markov Decision Process (MDP)** is a mathematical model used in Reinforcement Learning to make decisions in an environment. It consists of **states**, **actions**, **transition probabilities**, **rewards**, and a **policy**. The next state depends only on the current state and action, making it suitable for sequential decision-making.

Components of MDP

- **State (S):** Current situation of the agent.
- **Action (A):** Decision taken by the agent.
- **Reward (R):** Feedback after taking an action.
- **Transition:** Movement from one state to another.

- **Policy (π):** Rule used to select actions.

Algorithm

1. Define the states and actions.
2. Set the initial state.
3. Select an action based on the policy.
4. Move to the next state.
5. Receive the reward.
6. Repeat the process until the goal state is reached.
7. Display the state transitions and rewards.

Python Program

Simple Markov Decision Process Simulation

```
states = ["Home", "Office", "Goal"]
```

```
rewards = [0, 5, 10]
```

```
current_state = 0
```

```
print("Initial State:", states[current_state])
```

```
while current_state < len(states) - 1:
```

```
    current_state += 1
```

```
    print("Moved to:", states[current_state])
```

```
    print("Reward:", rewards[current_state])
```

```
print("Goal Reached!")
```

Sample Output

Initial State: Home

Moved to: Office

Reward: 5

Moved to: Goal

Reward: 10

Goal Reached!

Result

The Markov Decision Process was successfully simulated. The agent transitioned between states, received rewards, and reached the goal state by following the defined policy.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Introduction

The Bellman Equation is one of the fundamental concepts in Reinforcement Learning. It helps estimate the value of a state by considering the immediate reward and the expected future rewards. It is widely used in algorithms such as Value Iteration, Q-Learning, and Deep Q-Networks (DQN).

Bellman Equation

The Bellman Equation is:

$$V(s) = R + \gamma V(s')$$

Where:

- $V(s)$ = Value of the current state
- R = Immediate reward
- γ (Gamma) = Discount factor (0 to 1)
- $V(s')$ = Value of the next state

Algorithm

1. Define the reward values.
2. Set the discount factor (γ).
3. Assign a value to the next state.
4. Apply the Bellman Equation.
5. Calculate the current state value.
6. Display the result.

Python Program

Bellman Equation Demonstration

```
reward = 10
gamma = 0.9
next_state_value = 20

current_state_value = reward + gamma * next_state_value

print("Reward:", reward)
print("Discount Factor:", gamma)
print("Next State Value:", next_state_value)
print("Current State Value:", current_state_value)
```

Sample Output

```
Reward: 10
Discount Factor: 0.9
Next State Value: 20
Current State Value: 28.0
```

Explanation

- The immediate reward is 10.
- The discount factor is 0.9, which gives importance to future rewards.
- The next state's value is 20.
- Using the Bellman Equation, the current state's value is calculated as:

$$10 + (0.9 \times 20) = 28$$

Applications

- Robot Navigation
- Game Playing (Chess, Atari)
- Smart Home Automation
- Autonomous Vehicles
- Resource Allocation

Result

The Bellman Equation was successfully implemented to calculate the value of the current state by combining the immediate reward with the discounted future reward.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand